DPI343
Computer Organization and Assembly Language

Lecture 11
Timer interrupt. Programmable timer and sound

Lectures - prof. em. U. Sukovskis

Labs for local students - assoc. prof. P. Rusakovs

Labs for foreign studdents - assoc. prof. G. Alksnis

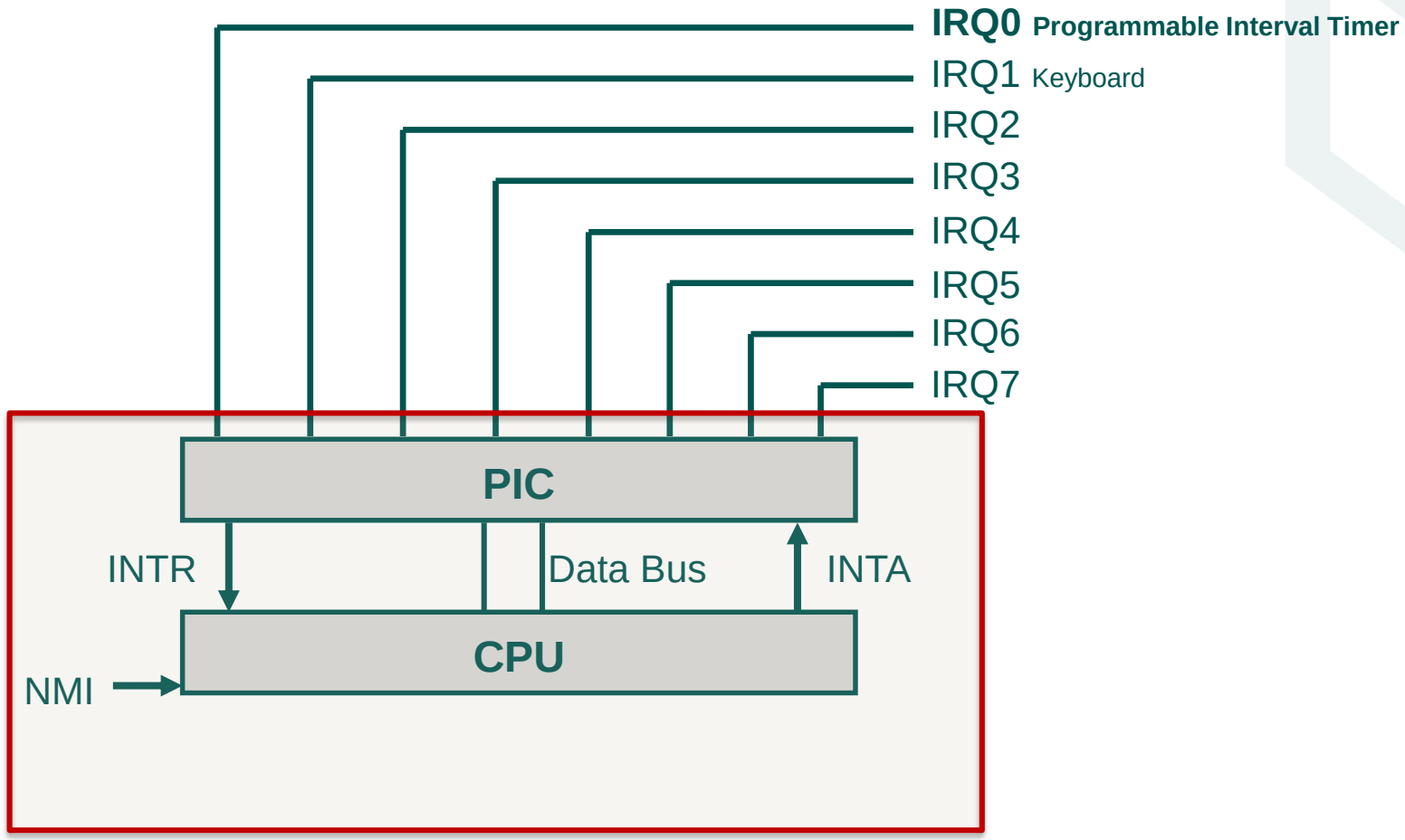
Autumn 2025

RTU
DATORZINĀTNES,
INFORMĀCIJAS
TEHNOLOĢIJAS
UN ENERĢĒTIKAS
FAKULTĀTE

Timer interrupt - basics

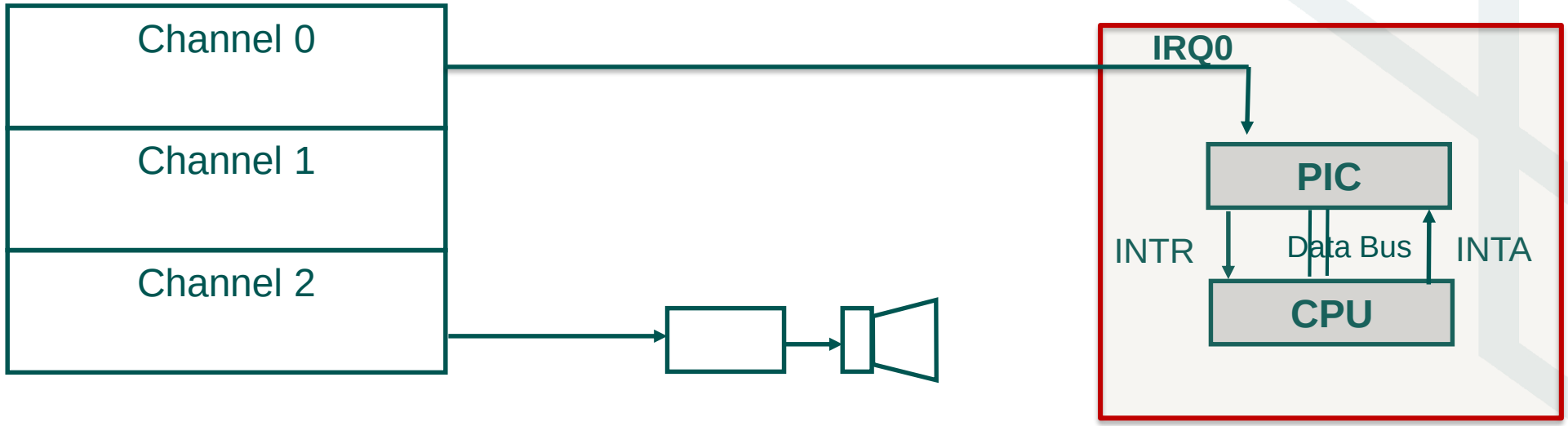
- **Timer interrupt** is a hardware interrupt (IRQ0) activated by the **Programmable Interval Timer** (Intel 8254) **18.2 times per second (every 55 ms).**
- Each timer interrupt normally is processed by the BIOS timer interrupt service routine which:
  - increments the System Timer Counter 4-byte integer value at 0000:046C by 1, i.e. one timer tick is 55 ms
  - executes instruction int 1Ch (so called User Timer Tick interrupt) . Default version of 1Ch interrupt handler contains only instruction IRET
- **Current time value is stored at address 0000:046C (System Timer Counter)** in the BIOS Data Area as an integer value (4 bytes) – the number of timer ticks
- After 24 hours of operation a flag is set at memory location 0000:0470 to signal this condition and the System Timer Counter is reset to 0
- System functions are available (by using int 21h interrupt) to convert this integer value to hours, minutes, seconds, and 1/100 s

PC Hardware Interrupts



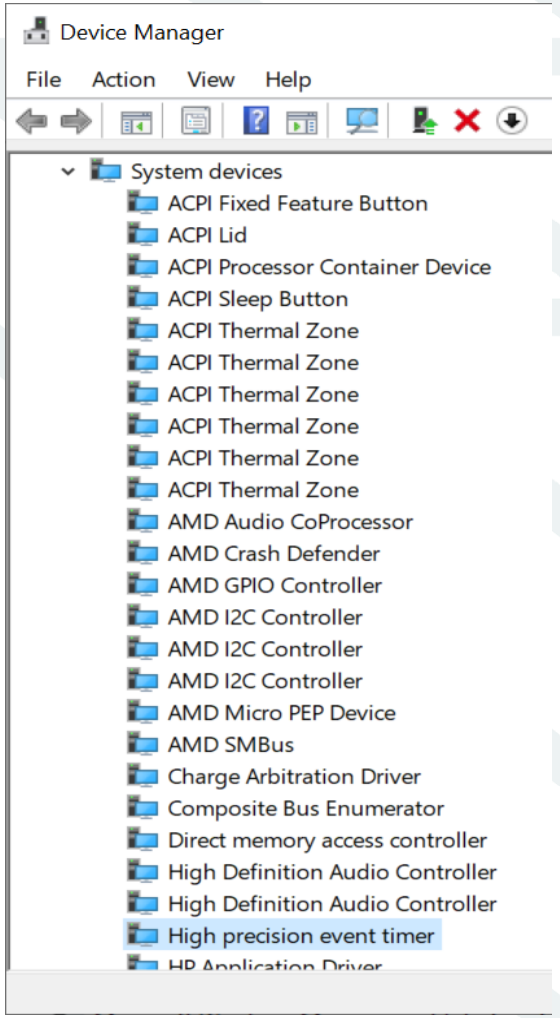
Intel 8254 Programmable Interval Timer (PIT)

PIT has 3 programmable channels, Channel 0 is connected to PIC, Channel 2 is connected to internal speaker



Timer interrupt - HPET

- In modern x86-compatible computer systems the **High Precision Event Timer (HPET)** is used instead of Programmable Interval Timer
- HPET has higher frequency and wider 64-bit counters, but operating principles are similar to PIT
- HPET allows more efficient processing of time sensitive multimedia applications and operating system task switching
- APIC (Advanced Programmable Interrupt Controller) also have buit-in timer capabilities used by some systems similarly as PIT



Example

Task

Write a timer interrupt handler to show a message after each 10 seconds in the middle of the text screen using different foreground and background colors

What we already know and what to consider

- **Programmable Interval Timer is connected to IRQ0 of the PIC**, i.e. the number of timer interrupt is **8**
- When timer interrupt occurs, our ISR code will be executed instead of the standard timer ISR
- Our own timer interrupt handler will be executed on each timer interrupt (**i.e. each 55ms**)
- **Our ISR must know the entry point address of the standard (i.e. previous) ISR** to be able to transfer control to it (to keep clock running which is based on System Timer Counter value)
- To set our own code as a interrupt handler we'll load our program into memory, **store the entry point address of the procedure into interrupt vector No. 8** (located at address 0000:0020)
- When this preparation is complete, the program will exit, but will **leave code of the interrupt handler procedure in memory**
- It would be good to **prevent multiple copies of our interrupt handler in the memory**. To do this, the startup procedure have to check weather the current timer interrupt handler is already our own code or not.

Example

To do:

A TSR structure program must be written, consisting of two parts:

1) startup procedure that prepares our ISR for operation
2) procedure that implements the ISR operation and will be called every time an interrupt occurs

In the startup procedure **main**:

- get and save entry point address of the current timer interrupt handler
- check if the current handler is not already our own
- set new timer interrupt vector, i.e. write the address of our handler's procedure entry point into the vector
- show message on screen that new handler is active
- exit but keep the handler procedure in memory

In the interrupt handler procedure **int8h**:

- save registers used in the procedure
- implement a counter to know when the 10-second limit has been reached
- if 10 seconds are elapsed, prepare and show the message on screen
- restore register values and transfer control to the original (previous) handler

Structure of the TSR source code

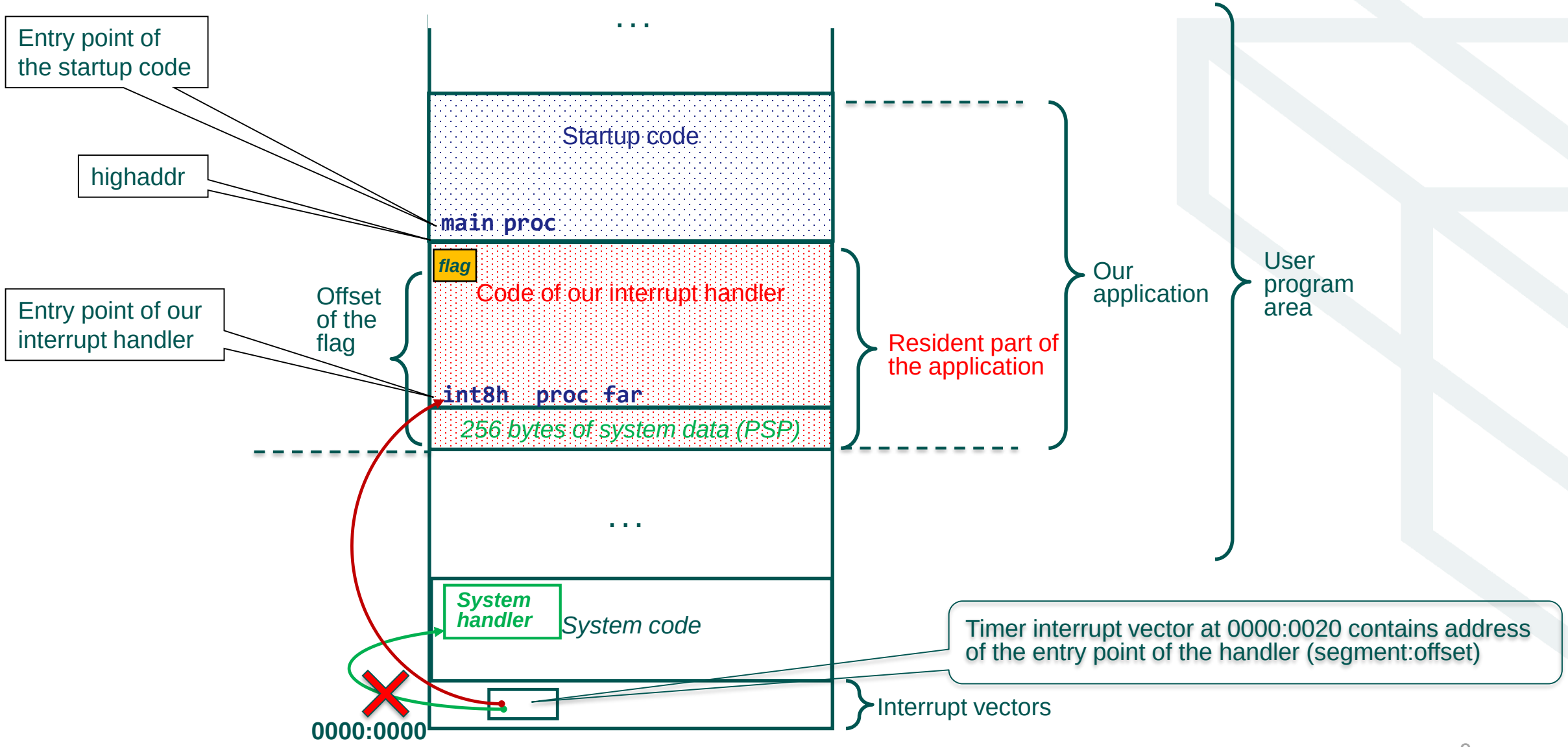
```
.model small
.code
int8h     proc far  ; interrupt 8 handler
          push es   ; save registers
          push ax
          :::       ; Check number of timer ticks and show the message
          pop  ax
          pop  es
          iret
int8h     endp
; Data used by the interrupt handler
:::
highaddr:

main      proc      ; Entry point of the startup code
          :::
;---------Get and save entry point address of the current handler
          :::
;---------Set new interrupt vector
          :::
;---------Show message on the screen that new handler is active
          :::
;---------Exit and keep the handler in memory
          :::
main      endp
end main
```

This part of the code remains in memory (ISR)

This part of the code is executed on startup

What does it look like in the memory?



Example

```
;============Interrupt handler procedure=======
int8h     proc    far
          push es             ; save registers
          push ax
          push di
          sti                 ; set interrupt flag
          inc  cs:color       ; change colors on each tick
;-------Check time interval
          inc  cs:ticks       ; data is stored in the code segment, so use cs for addressing
          cmp  cs:ticks, 182  ; 182 ticks (10 seconds) elapsed?
          jl   retold         ; if less then transfer control to the previous ISR
          mov  cs:ticks, 0    ; else set tick counter to zero
;-------The time interval elapsed so write Hi! in the middle of the screen
          mov  ax, 0B800h     ; segment value of the text mode video memory
          mov  es, ax
          ;...                ; write text and attributes bytes to video memory
;-------Transfer control to the previous ISR
retold:   pop  di         ; restore registers
          pop  ax
          pop  es
          jmp  cs:[oldint8]  ; transfer control
int8h     endp
;--------Data used by interrupt handler
oldint8   dword    0              ; saved entry point address of the previous handler
ticks     word     0              ; ticks elapsed
color     byte     00100111b      ; text and background colors
flag      byte     'MYTIMER'      ; a flag to prevent multiple copies of the handler
highaddr:                         ; Keep the code in memory up to this address
```

Example cont.

```
;===============Startup code============================
;--------Data used by the startup code
msgok    byte      'TimerHi activated',13,10,'$'
msgerr   byte      'TimerHi is already active!',13,10,'$'

main proc              ; Entry point of the startup code
         push cs       ; Data is stored in the code segment
         pop  ds       ; so copy cs to ds
;----Prevent duplicate copies of the same interrupt handler in memory
         ; . . . some code to check the flag location in the currently active ISR
activate:
         mov  dx, offset msgInit
         mov  ah, 9
         int  21h
;---------Get and save the entry point address of the currently active ISR
         mov  ax, 0
         mov  es, ax                ; segment 0
         mov  bx, es:[4*8]          ; offset value from interrupt vector
         mov  word ptr oldint8, bx  ; save offset value of the entry point address of the active ISR
         mov  ax, es:[4*8+2]        ; segment value from interrupt vector
         mov  word ptr oldint8+2, ax    ; save segment value of the entry point address of the active ISR
;---------Set new interrupt vector - entry point address of our handler int8h
         mov  es:[4*8], offset int8h    ; entry point offset of the new interrupt handler
         mov  ax, cs
         mov  es:[4*8+2], ax            ; entry point segment of the new interrupt handler
;---------Show message that our timer interrupt handler is active
```

Example cont.

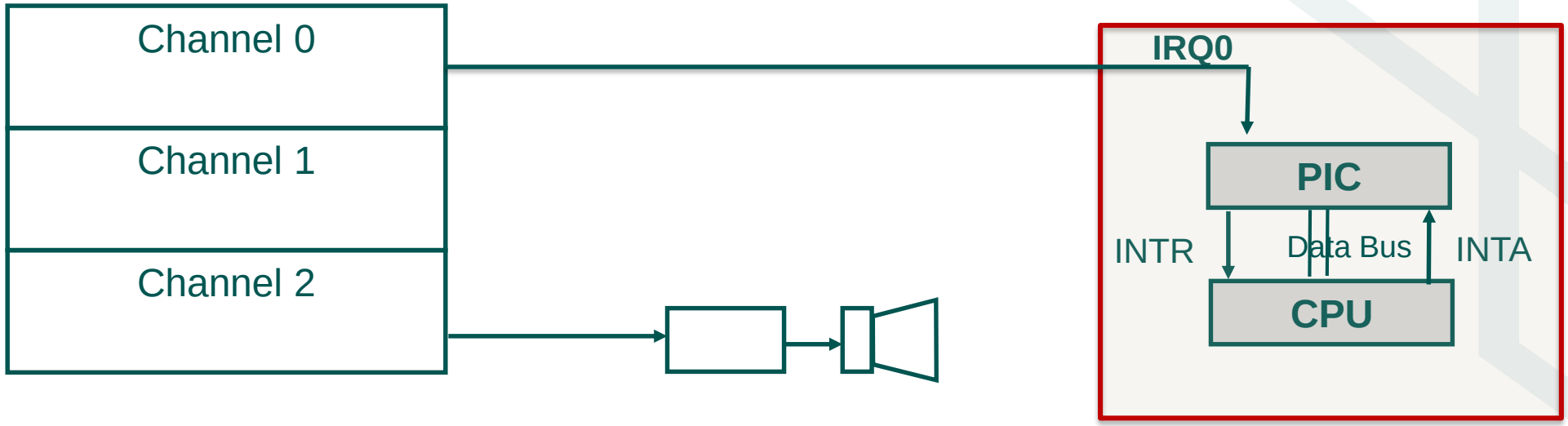
```
;---------Show message that our timer interrupt handler is active
        mov dx, offset msgok
        mov ah, 9
        int 21h
;---------Exit and keep our handler code in memory
        mov dx, offset highaddr
        add dx, 10Fh          ; add the size of the PSP and round up to the paragraph
        shr dx, 4             ; divide by 16
        mov ax, 3100h         ; function 31h, return code 00h
        int 21h               ; return to the system and keep the code of our ISR in memory
main    endp

end main
```

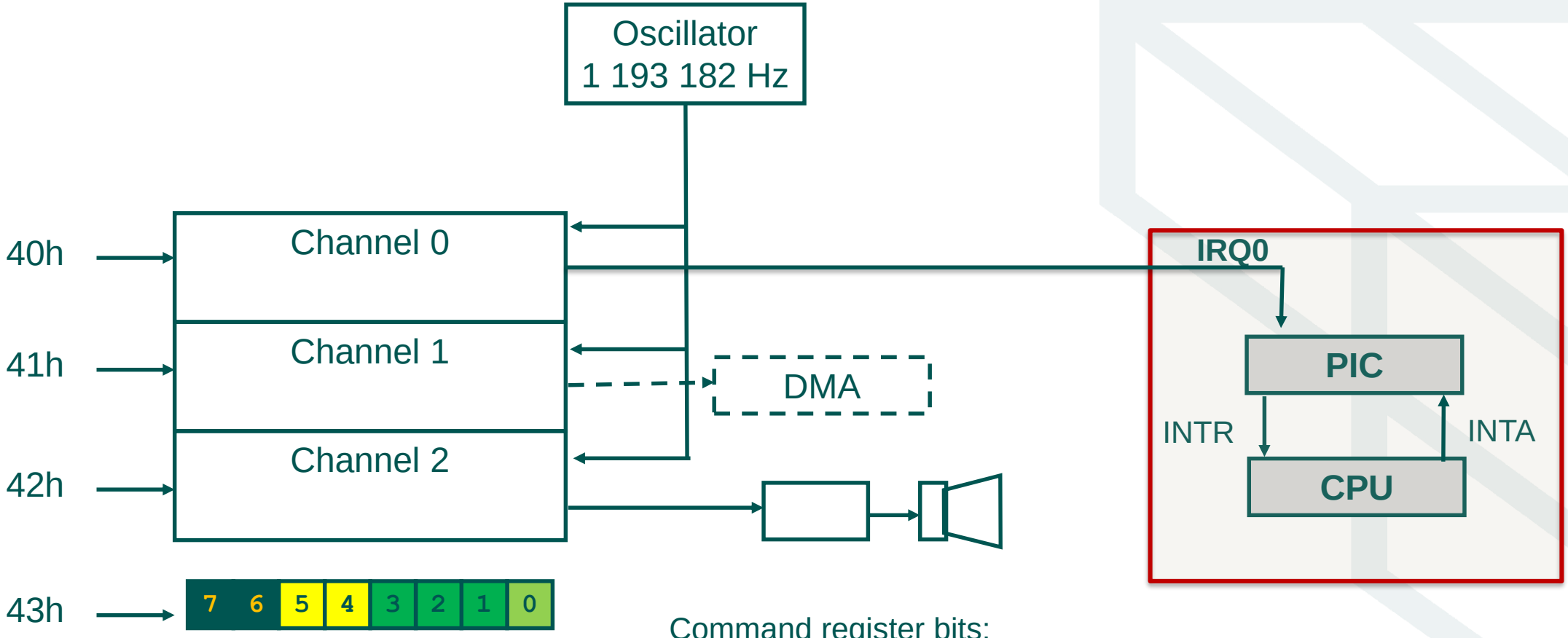
Programmable Interval Timer and sound

Intel 8254 Programmable Interval Timer (PIT)

PIT has 3 programmable channels, Channel 0 is connected to PIC, Channel 2 is connected to internal speaker



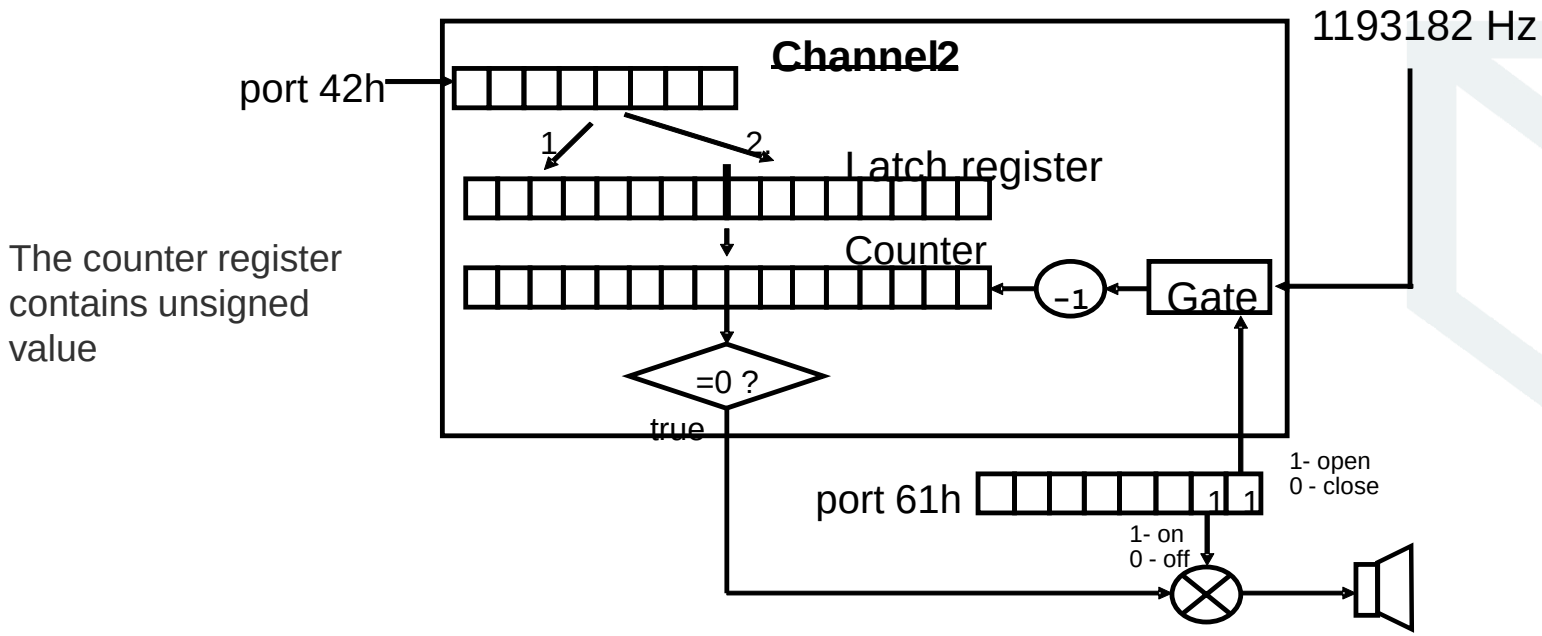
Intel 8254 Programmable Interval Timer (PIT)



Command register

Command register bits:
7, 6 - channel number
5, 4 - bytes to load in the latch register
3, 2, 1 - mode of operation
0 – format of the counter value 0=bin or 1=dec

PIT Channel 2



For the mode of operation 3 each input impulse from the oscillator decrements the counter register by 1.

When the counter has reached 0 the output is generated and the counter is reset again to the initial value from the latch register.

For example, if the initial counter value is 100 the output impulse will be generated after each 100 input impulses, i.e. output frequency will be 1193182 / 100 ≈ 11932 Hz (Output = Input/Counter).

To calculate counter value to have output frequency required you may use a formula:

Counter value = Input frequency / Output frequency

PIT Programming Example

Task

Write a code to generate 1000 Hz sound for a few seconds using Channel 2 of the Programmable Interval Timer

To do:

- Set command register bits to access Channel 2 and to provide the desired settings
- Calculate value of the counter register needed to generate 1000 Hz sound
- Set the counter register value
- Open both gates to start sound
- Do something for few seconds
- Close gates to stop sound

PIT Programming Example


```
mov   al, 10110110b    ; 10-channel 2, 11 - two bytes, 011 – mode=3, 0 - bin
out   43h, al          ; set command register

mov   ax, 1193         ; counter=1193182/1000 (to generate 1000 Hz beep)
out   42h, al          ; write to the Latch register low order byte
mov   al, ah           ; and then
out   42h, al          ; high order byte

in    al, 61h          ; read port 61h
or    al, 00000011b    ; enable gate and speaker
out   61h, al          ; start beeping

:::                    ; some code to run for a few seconds

in    al, 61h          ; read port 61h
and   al, 11111100b    ; set bits to 0 to close gates
out   61h, al          ; stop beeping
```